Reranking and Retrieval Refinement

pig7selene

September 5, 2026

Abstract

First-stage retrieval must search a large corpus quickly, so it is usually designed to preserve useful evidence at a generous candidate depth. A reranker makes a more selective comparison over that bounded set before context construction. This chapter develops the recall–precision division of labor, contrasts Bi-Encoder retrieval with joint Cross-Encoder scoring, and explains pointwise, pairwise, listwise, and LLM-based reranking. It also treats diversity, metadata, cascades, latency, and operational contracts as parts of the ranking objective rather than afterthoughts.

1 Introduction

The retrieval pipeline developed in Chapters 27–30 deliberately separates broad search from fine discrimination. A dense, sparse, or hybrid retriever must compare a query with a corpus that may contain millions of retrieval units. Its score is therefore designed for a representation that can be indexed or precomputed. The resulting first-stage list is useful only as a **candidate set**: it is an affordable opportunity for a later component to make a more informed relevance judgment.

Reranking is that second-stage judgment. The full pipeline has a clear division of labor:

query → sparse / dense / hybrid retrieval → candidate set
→ reranker → final ranked passages → context construction → generator

The first stage should retain answer-supporting evidence even when its ranking is imperfect. The reranker is then allowed to spend more computation on a modest number of candidates to put the most useful evidence near the top. Final **context selection** is still a separate decision: it turns ranked passages into a bounded, nonredundant prompt with provenance and formatting compatible with the generator. Conflating candidate generation, reranking, and context selection makes it difficult to identify whether an unsupported answer began with a retrieval miss, a ranking error, or a poor prompt assembly decision.

This chapter focuses on retrieval refinement. It does not develop query rewriting, multi-query retrieval, HyDE, iterative retrieval, graph-based retrieval, agentic RAG, or a complete RAG evaluation framework. Those methods can alter the candidate interface considered here, but they do not remove the need to state what a reranker scores and what evidence reaches it.

2 Candidate Generation, Recall, and Candidate Depth

For a query q, let the first-stage retriever return an ordered candidate set

$$C_{K(q)} = (d_1, ..., d_K), \tag{1}$$

where K is the **candidate depth**. Let $G(q)$ denote the set of judged relevant units for the query. A simple candidate-recall measure is

$$\text{Recall}@K(q) = \frac{|C_{K(q)} \cap G(q)|}{|G(q)|}. \tag{2}$$

The exact relevance judgments and averaging convention must be stated for a real evaluation, but Equation 2 exposes the structural constraint: no reranker can recover a relevant passage that is absent from $C_{K(q)}$. First-stage Recall@K is therefore an upper bound on the evidence available to a second stage. A reranker can repair an order; it cannot repair a candidate-generation miss.

This explains why first-stage retrieval is commonly recall-oriented. Chapters 27 and 28 showed that a Bi-Encoder and an approximate-nearest-neighbor index exchange exact comparison for scalable search. Chapter 30 showed that sparse and dense paths can expand the set through hybrid candidate generation. These designs may leave related, partially redundant, or merely topical passages in $C_{K(q)}$. That is acceptable when the next stage can distinguish them. A first stage tuned only for its own Precision@K can instead discard the one source that contains the condition, date, or counterexample the generator needs.

Candidate depth is a system parameter, not a harmless constant. If K is too small, first-stage recall constrains every later component. If K is too large, joint scoring becomes expensive, creates more duplicate candidates, and can delay an interactive response. The appropriate value depends on corpus size, query ambiguity, retrieval-unit granularity from Chapter 29, reranker cost, context budget, and the desired tail latency. It should be selected with end-to-end evidence rather than copied from a benchmark configuration.

3 Bi-Encoders and Cross-Encoders

Chapter 27 introduced a Bi-Encoder, or Dual-Encoder, in which a query encoder and a document encoder produce independent vectors. The system can precompute document vectors, index them, and score a query with a similarity function. This independent encoding is exactly why a Bi-Encoder scales to a large corpus. It is also a restriction: the document representation cannot condition on the particular query at scoring time.

A **Cross-Encoder** jointly processes the query and one candidate document. Conceptually,

$$(q, d) \rightarrow \text{joint Transformer} \rightarrow r_{\theta(q,d)}, \tag{3}$$

where $r_{\theta(q,d)}$ is a learned relevance score. Because the Transformer's attention can compare query tokens with document tokens directly, the model can recognize a qualifying phrase, a negation, an identifier, or a relation whose importance depends on both inputs. BERT-style passage reranking was an early influential demonstration of applying a jointly encoded pretrained Transformer to query–passage ranking [1].

The same interaction prevents precomputing one reusable score or embedding for every possible query. A Cross-Encoder must run its forward pass for each (q, d) pair, so applying it to the whole corpus would be far more expensive than first-stage vector or lexical retrieval. The two architectures are therefore complements rather than substitutes:

Component	Representation and comparison	Operational role
Bi-Encoder retriever	Encode q and d independently; compare reusable vectors or sparse features	Broad corpus search and high-recall candidate generation
Cross-Encoder reranker	Jointly encode a query–candidate pair; score token-level interactions	Precision-oriented ordering over a bounded candidate set
Context selector	Select, deduplicate, and format highly ranked evidence	Fit useful, attributable evidence into the generator's context budget

Table 1: Candidate generation, reranking, and context selection use different representations and optimize different interfaces. A high Cross-Encoder score does not itself decide how many passages should enter a prompt.

Cross-Encoder scores are ranking signals, not automatically calibrated probabilities of truth, usefulness, or user satisfaction. Their numerical scale depends on the training labels, objective, candidate distribution, and model revision. A deployment should preserve the score for observability, but should not treat a larger raw score as comparable across unrelated model versions or query populations without validation.

4 Ranking Objectives

Learning to rank can be organized by the object on which supervision is expressed: an individual item, a pair of items, or a whole ordered list [2]. The categories clarify what a reranker is trained to preserve; they do not prescribe one universally superior architecture.

In a **pointwise** formulation, the model predicts a relevance value for a single candidate, such as $r_{\theta(q,d)}$. Labels may be binary, graded, or derived from interaction data. Pointwise scoring is straightforward to serve because candidates can be scored independently, but the objective need not directly express whether one candidate should be above another for the same query.

In a **pairwise** formulation, supervision states a relative preference. For candidates d_i and d_j, a standard conceptual model is

$$P\big(d_i \succ d_j \texttt{ mid } q\big) = \sigma\Big(r_{\theta(q,d_i)} - r_{\theta(q,d_j)}\Big). \tag{4}$$

The corresponding negative log-likelihood increases the score gap when d_i should rank above d_j. Pairwise supervision matches the comparative nature of ranking, but the number of possible pairs grows rapidly with candidate-set size. It also requires a policy for ties, incomplete judgments, and pairs that are equally relevant but differ in source authority or freshness.

Listwise approaches optimize properties of an ordered candidate list or a distribution over permutations. They can better align training with a ranking metric that values the placement of several items, but they commonly require more structured labels and more complex training or serving behavior. The important design question is not which label is fashionable; it is whether the supervision and loss represent the downstream intent.

Formulation	Supervision unit	Principal trade-off
Pointwise	A query–candidate relevance label	Simple independent scoring; relative ordering is only indirect
Pairwise	A preferred candidate relative to another candidate	Direct comparative signal; pair construction and cost matter
Listwise	A candidate list or its desired ordering	Closer to list-level ranking goals; labels and optimization are more involved

Table 2: Pointwise, pairwise, and listwise objectives differ in the unit of supervision. Their suitability depends on the available labels and the final ranking criterion, not only on model capacity.

These distinctions apply to both traditional learned rankers and neural rerankers. A Cross-Encoder is an architecture for producing a query-conditioned score; it can be trained under different ranking objectives. Conversely, a pointwise score can be used in a broader list-level selection policy. Keep the modeling objective, score meaning, and final selection rule separate in the system specification.

5 Precision, Usefulness, and Context Selection

Reranking is often described as improving **precision**, but relevance itself is multidimensional. A passage can be topically related yet fail to answer the question. It can contain an answer but be too incomplete to support a grounded response. It can be useful evidence but obsolete, unauthorized, or less authoritative than an available primary source. A RAG pipeline should consequently distinguish topical relevance, answer usefulness, evidence quality, source authority, freshness, and eligibility.

The distinction becomes concrete at context construction. Suppose the reranker places five near-duplicate passages from a single source above a complementary source containing a missing assumption. A context constructor that blindly takes the top five can reduce answer quality despite a locally accurate ranker. It may need to suppress duplicates, preserve source identity, prefer a more current document, or select one detailed passage and one independent corroborating passage. These are selection decisions over the reranked list, not reasons to hide metadata from the ranker.

Maximal Marginal Relevance (MMR) is a representative diversity-aware rule. Given already selected passages S, it chooses a candidate that balances query relevance and similarity to what is already selected:

$$\mathrm{MMR}(d \mid S) = \lambda r(q, d) - (1 - \lambda) \max_{u \in S} \mathrm{sim}(d, u), \quad 0 \le \lambda \le 1. \tag{5}$$

The first term prefers evidence relevant to q; the second penalizes redundancy relative to S. When λ is large, selection approaches relevance-only ranking. When it is small, diversity has more influence. MMR is a useful conceptual mechanism, not a guarantee that dissimilar passages are useful. An unrelated result is diverse but does not improve evidence coverage. The original MMR formulation made this relevance–novelty trade-off explicit for document reordering and summarization [3].

Metadata-aware refinement supplies additional constraints or features. Date, document version, source type, language, tenant, permission, and authority can be eligibility filters, ranking features, or tie-breakers. The policy should state which role each field plays. A hard permission restriction must never become a soft relevance preference; a freshness preference should not silently override a source's technical authority. Chapter 28′s filtering discussion applies here: a ranking result is meaningful only over the eligible corpus actually considered.

6 LLM-Based Reranking and Retrieval Cascades

An LLM can also make query-conditioned relevance judgments. In a **pointwise** prompt, it assigns a label or score to one query–passage pair. In a **pairwise** prompt, it chooses which of two candidates better answers the query. In a **listwise** prompt, it orders several candidates together. Such prompting can express nuanced criteria, including whether a passage contains usable evidence rather than merely a shared topic. Research on LLMs as reranking agents demonstrates that appropriately prompted generative models can be evaluated directly on relevance ranking, while also exposing concerns around novel-data evaluation and ranking cost [4].

The flexibility comes with constraints. Each LLM judgment consumes tokens and introduces latency that may exceed a specialized Cross-Encoder. Its ranking can vary with prompt wording, candidate order, truncation, decoding configuration, and the judge model's own capabilities. Position bias can reward early candidates; verbosity bias can mistake a longer passage for a more useful one; context limits can force the system to omit information needed for a fair comparison. An LLM reranker should therefore be evaluated as a model with a declared prompt, decoding configuration, candidate order policy, and cost budget, not as an oracle.

These trade-offs motivate a **retrieval cascade**:

$$\begin{gathered}\text{large corpus} \longrightarrow \text{cheap retriever} \longrightarrow \text{broad candidate set} \\ \longrightarrow \text{stronger reranker} \longrightarrow \text{smaller set} \longrightarrow \text{optional expensive selector} \\ \longrightarrow \text{final context}\end{gathered}$$

An LLM need not appear in every cascade. A sparse, dense, or hybrid first stage followed by a Cross-Encoder is often the appropriate balance. An optional LLM stage may be reserved for ambiguous, high-value, or difficult requests, provided its added cost and nondeterminism are measured. The end-to-end latency is approximately

$$T_{\text{answer}} = T_{\text{retrieve}} + T_{\text{rerank}} + T_{\text{generate}}. \tag{6}$$

The terms in Equation 6 can overlap in a particular serving implementation, but the decomposition remains useful for diagnosis. Reducing retrieval latency cannot compensate for a reranker whose candidate depth or sequence length dominates time to first token. Conversely, skipping reranking may save milliseconds while causing the generator to process more irrelevant context and produce a less supported answer.

7 Metrics and Failure Diagnosis

Reranking should be evaluated at both its own interface and the final system interface. Recall@K asks whether first-stage retrieval exposed relevant evidence. Precision@k asks what fraction of a small displayed or selected prefix is relevant. **Mean Reciprocal Rank** (MRR) rewards placing the first relevant item early. **Normalized Discounted Cumulative Gain** (nDCG) can use graded relevance and discounts results that occur later in the list. These metrics answer different questions; reporting one number without candidate depth, judgment policy, filters, or latency hides the actual ranking trade-off.

A practical diagnostic separates three failure classes. A **candidate-generation failure** means answer-supporting evidence never reached $C_{K(q)}$. A **reranker failure** means the evidence was present but placed too low, confused with a more topical passage, or assigned a stale or poorly calibrated score. A **context-selection failure** means an adequately reranked passage was removed, crowded out by redundant items, truncated, or formatted so that the generator could not use it. This decomposition avoids blaming the embedding model for an MMR policy or blaming a Cross-Encoder for a sparse-retrieval miss.

Common failures can cross these boundaries. A domain-mismatched reranker can learn lexical shortcuts rather than evidence quality. An overly shallow K prevents recovery; an overly deep K raises latency and may exceed a cross-encoder's input budget. Duplicate passages can absorb ranking positions. A model can prefer verbose text, stale sources, or an early candidate position. Score scales can shift after a model update, making an old threshold invalid. The remedy is an observable trace: query revision, candidate IDs and ranks from each first-stage path, reranker inputs and scores, metadata decisions, selected context, and the model versions responsible for each step.

8 Implementation Contracts

The **candidate contract** must identify the corpus revision, retrieval-unit schema, eligibility filters, sparse and dense paths where used, candidate depths, deduplication rule, and the exact order passed to the reranker. It must distinguish a candidate identifier from a source-document identifier, so that repeated chunks and parent expansions remain observable. It should also define fallback behavior when no candidate is eligible or a first-stage service is unavailable.

The **reranking contract** must name the model, tokenizer, query–document serialization, truncation policy, maximum pair length, objective or prompt, score interpretation, batching policy, tie rule, and model revision. A Cross-Encoder must receive the same declared field layout at evaluation and serving time. An LLM-based reranker additionally needs a prompt template, candidate-order policy, decoding configuration, parser for its output, retry policy, and a budget that bounds tokens and latency.

The **selection contract** must specify how the ranked list becomes generator context: final depth, diversity rule, duplicate policy, metadata or authority constraints, source-attribution format, context budget, and treatment of conflicting evidence. The **evaluation contract** should report first-stage recall, reranker ranking metrics, selected-context support coverage, downstream answer quality, tail latency, cost, and slice results for exact identifiers, paraphrases, fresh sources, permissions, duplicate passages, and ambiguous questions. Version all judgments and queries so that a change in corpus, retriever, or reranker can be localized rather than attributed to a generic RAG regression.''

9 Summary

Reranking refines a bounded first-stage candidate set before evidence enters the generator. The first stage is optimized to make relevant evidence available; a Cross-Encoder or LLM-based reranker can then make a stronger query–passage comparison, at a cost that grows with candidate depth and input length. Pointwise, pairwise, and listwise objectives describe different forms of ranking supervision, while their downstream value depends on the relevance criterion they encode.

The final goal is not an abstract relevance score. It is a compact context containing useful, attributable, eligible, and nonredundant evidence. Diversity-aware selection, metadata policy, and cascaded latency budgets therefore belong to the retrieval design. A debuggable system separately measures candidate generation, reranking, and context selection, preserving the interfaces needed to improve one stage without obscuring a failure in another.

References

[1] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019, doi: 10.48550/arXiv.1901.04085.

[2] T.-Y. Liu, "Learning to Rank for Information Retrieval," *Foundations and Trends in Information Retrieval*, vol. 3, no. 3, pp. 225–331, 2009, doi: 10.1561/1500000016.

[3] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, Association for Computing Machinery, 1998, pp. 335–336. doi: 10.1145/290941.291025.

[4] W. Sun *et al.*, "Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 14918–14937. doi: 10.18653/v1/2023.emnlp-main.923.